

Signal Check

A mock Round 1 for the Voice AI hackathon track

Twelve concept MCQs and five CodeChef-style coding problems on speech recognition, text-to-speech, audio processing and conversational AI. Not affiliated with CodeChef -- a practice instrument built for the rvhack team. An interactive, timed version of this same test (with a live scoring/review screen) is linked from the project README.

DURATION	SECTIONS	QUESTIONS	TOTAL MARKS
60:00	2	17	100

How to use this

- Section A -- 12 concept MCQs, 5 marks each (60 total). One correct option per question, no negative marking.
- Section B -- 5 coding problems, 8 marks each (40 total), in CodeChef style: statement, constraints, sample input/output, with an approach and a reference Python solution.
- Read every explanation, even for questions answered correctly -- the reasoning is the point, not the score.
- Try the coding problems on paper or in an editor before reading the approach. Self-time each one; the real round is timed per-problem implicitly by the overall clock.

Section A -- Concept MCQs

60 marks total · 5 marks each · ASR, TTS, audio processing, conversational AI

Q1	ASR	5 MARKS
----	-----	---------

In Word Error Rate (WER), the denominator (N) used in $(S+D+I)/N$ is:

- A. The total number of words in the hypothesis (ASR output)
- B. The total number of words in the reference (ground-truth) transcript (correct)**
- C. The total number of characters in the reference
- D. The length of the audio clip, in seconds

Why: $WER = (Substitutions + Deletions + Insertions) / N$, where N is the number of words in the reference transcript, not the hypothesis. This is exactly why WER can exceed 100%: a hypothesis with many extra inserted words relative to a short reference racks up more errors than there are reference words.

Q2

ASR

5 MARKS

A 'final' transcript segment in a streaming ASR system differs from an 'interim' one because it:

- A. Is always longer than the interim version
- B. Will not be re-decoded or revised as more audio arrives (correct)**
- C. Is computed using a smaller acoustic model
- D. Always carries a confidence score above 0.9

Why: Streaming ASR emits low-latency 'interim' hypotheses that can change as more context arrives. A 'final' segment is the decoder's committed output for that stretch of audio and will not be re-decoded, even though later segments may still depend on it for context.

Q3

ASR

5 MARKS

Connectionist Temporal Classification (CTC) lets an acoustic model output a shorter text sequence than the number of input frames mainly by:

- A. Running beam search over multiple language models
- B. Introducing a blank/null token and collapsing repeated labels (correct)**
- C. Requiring exact frame-to-character alignment labels during training
- D. Skipping every alternate input frame

Why: CTC introduces a special blank symbol plus a collapsing rule (merge repeated labels, drop blanks), so the network can emit one output per frame yet still produce a much shorter label sequence, without needing frame-level alignment during training.

Q4

AUDIO

5 MARKS

A signal contains frequency components up to 8 kHz. Per the Nyquist-Shannon theorem, the minimum sampling rate that avoids aliasing is:

- A. 4 kHz
- B. 8 kHz
- C. 16 kHz (correct)**
- D. 32 kHz

Why: To represent a signal with frequencies up to f_{max} without aliasing, you must sample at $\geq 2 \times f_{max}$. For $f_{max} = 8 \text{ kHz}$, the minimum rate is 16 kHz.

Q5

AUDIO

5 MARKS

The primary purpose of a Voice Activity Detector (VAD) in a voice pipeline is to:

- A. Transcribe speech into text
- B. Distinguish speech frames from silence/noise so downstream stages only process speech (correct)**
- C. Cancel acoustic echo from the speaker output
- D. Convert PCM audio into the Opus codec

Why: A VAD classifies audio as speech vs. non-speech. That lets the pipeline skip sending silence to expensive downstream stages (ASR, LLM) and helps decide when the user has actually finished talking -- endpointing.

Q6

CONVERSATIONAL AI

5 MARKS

In a voice agent, 'barge-in' refers to:

- A. The system interrupting the user to ask a clarifying question
- B. The user interrupting the assistant's speech mid-response, which must stop TTS and cancel/truncate the in-flight reply (correct)**
- C. Two speakers overlapping on a call, causing a diarization error
- D. A fallback mode triggered when STT confidence is low

Why: Barge-in is the user cutting into the assistant's spoken response. Handling it correctly means immediately stopping TTS playback and cancelling/truncating the in-flight generation, so conversation history reflects only what the user actually heard.

Q7

TTS

5 MARKS

Which metric specifically measures how long a user waits before hearing ANY audio after a TTS request, not the time to synthesize the full utterance?

- A. Mean Opinion Score (MOS)
- B. Word Error Rate (WER)
- C. Time-to-first-byte / first-audio latency (correct)**
- D. Real-time factor (RTF)

Why: Time-to-first-byte (first-audio latency) measures how quickly playback can begin -- what a real-time conversation actually feels like. Streaming TTS exists specifically to optimize this number.

Q8

TTS

5 MARKS

Before phoneme conversion, a TTS front-end expands the currency amount 'Rs 2,450' into a spoken form. This step is called:

- A. Diarization
- B. Text normalization (correct)**
- C. Prosody prediction
- D. Vocoding

Why: Text normalization expands non-orthographic tokens -- currency, dates, abbreviations, numerals -- into words a phonemizer can convert to pronunciation, before prosody and vocoding happen.

Q9

AUDIO

5 MARKS

Mu-law (G.711) encoding is used in traditional telephony because it:

- A. Provides lossless compression at 8 kHz
- B. Is a companding scheme that non-linearly maps 16-bit audio into 8 bits, giving better perceptual quality per bit for speech than linear PCM at the same bit depth (correct)**
- C. Removes background noise automatically
- D. Is a neural vocoder

Why: Mu-law/A-law are logarithmic companding schemes: they allocate more of the 8-bit codeword range to quiet sounds, where the ear is more sensitive, giving far better perceptual SNR for speech than naive 8-bit linear PCM at the same bit rate.

Q10

CONVERSATIONAL AI

5 MARKS

In an LLM-based voice agent, 'tool calling' (function calling) is primarily used to:

- A. Reduce the number of tokens in the prompt
- B. Let the model request an external action/API call and receive a structured result to continue reasoning (correct)**
- C. Automatically translate the user's spoken language
- D. Improve the acoustic model's transcription accuracy

Why: Tool/function calling lets the LLM emit a structured request (name + arguments) for code outside the model to execute -- a lookup, a booking, a database write -- then feeds the result back in, grounding the response in real data instead of a guess.

Q11

CONVERSATIONAL AI

5 MARKS

Speaker diarization answers which question about a recording?

- A. What was said?
- B. Who spoke, and when, in a multi-speaker recording? (correct)**
- C. How loud was the speaker?
- D. Which language was spoken?

Why: Diarization segments and clusters audio by speaker identity -- 'who spoke when' -- independently of what was said (ASR's job) or which language was used.

Q12

ASR

5 MARKS

OpenAI's Whisper is often described as not natively 'streaming' because it:

- A. Requires an internet connection to run
- B. Processes audio in fixed ~30-second windows via a sequence-to-sequence encoder-decoder rather than emitting partial results frame-by-frame (correct)**
- C. Only supports the English language
- D. Has no attention mechanism

Why: Whisper's encoder-decoder consumes a full (padded) 30-second mel-spectrogram window and autoregressively decodes text. It has no built-in mechanism to emit partial tokens as audio arrives -- 'streaming Whisper' setups are short-window chunking wrapped around a fundamentally batch model.

Section B -- Coding Problems

40 marks total · 8 marks each · solve, then compare against the reference approach

Problem 1

EASY

8 MARKS

WER-1: Word Error Rate

You're given a reference transcript REF and a hypothesis transcript HYP, each a sequence of space-separated uppercase words. Compute the Word Error Rate using standard Levenshtein alignment over words: $WER = (\text{edit_distance}(\text{REF_words}, \text{HYP_words}) / \text{len}(\text{REF_words})) \times 100$, rounded to 2 decimal places.

CONSTRAINTS

1 ≤ number of words in REF, HYP ≤ 1000. Words contain only uppercase letters A-Z.

SAMPLE INPUT

REF: I WENT TO THE STORE TODAY
HYP: I GO TO STORE TODAY

SAMPLE OUTPUT

33.33

APPROACH

Treat each word as one DP token and run classic Levenshtein over the two word sequences (not characters). $dp[i][j]$ = min edits to turn the first i reference words into the first j hypothesis words. The answer is $dp[N][M] / N \times 100$.

REFERENCE SOLUTION (PYTHON)

```
def wer(ref, hyp):
    r, h = ref.split(), hyp.split()
    n, m = len(r), len(h)
    dp = [[0]*(m+1) for _ in range(n+1)]
    for i in range(n+1): dp[i][0] = i
    for j in range(m+1): dp[0][j] = j
    for i in range(1, n+1):
        for j in range(1, m+1):
            if r[i-1] == h[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return round(dp[n][m] / n * 100, 2)
```

Problem 2

MEDIUM

8 MARKS

VAD-1: Frame Energy Speech Segments

You're given N integer PCM samples of a mono signal, a frame size F and an energy threshold T . Split the samples into consecutive non-overlapping frames of exactly F samples (discard a trailing partial frame). For frame k (0-indexed), $\text{energy}(k) = \text{sum of squares of its samples}$; mark it SPEECH if $\text{energy}(k) > T$, else SILENCE. Merge consecutive SPEECH frames into maximal segments and print each as 'start end' (inclusive frame indices), one per line. If there is no SPEECH frame, print NONE.

CONSTRAINTS

1 ≤ N ≤ 10^5 . 1 ≤ F ≤ N . 0 ≤ T ≤ 10^{12} . Samples fit in a 32-bit signed integer.

SAMPLE INPUT

```

samples = 0 0 50 60 -40 0 0 0 90 -90 100 0 0
0
F = 2
T = 3000

```

SAMPLE OUTPUT

```

1 1
4 5

```

APPROACH

Walk the samples in steps of F , sum of squares per frame to get a boolean `is_speech` array, then one linear scan merging consecutive True runs into `[start, end]` pairs.

REFERENCE SOLUTION (PYTHON)

```

def vad_segments(samples, F, T):
    n = len(samples)
    frames = n // F
    is_speech = []
    for k in range(frames):
        chunk = samples[k*F:(k+1)*F]
        energy = sum(x*x for x in chunk)
        is_speech.append(energy > T)
    segments, i = [], 0
    while i < frames:
        if is_speech[i]:
            j = i
            while j+1 < frames and is_speech[j+1]:
                j += 1
            segments.append((i, j))
            i = j + 1
        else:
            i += 1
    return segments # print "NONE" if empty

```

Problem 3**EASY-MEDIUM****8 MARKS****MERGE-1: Word Timestamps to Utterances**

You're given N word-level timestamps as pairs `(start_i, end_i)` in milliseconds, sorted by start, describing consecutive non-overlapping spoken words. Given a pause threshold P , merge words into utterances: two consecutive words belong to the same utterance if `start[i+1] - end[i] ≤ P`; otherwise a new utterance begins. Print each utterance as 'start end' on its own line, in order.

CONSTRAINTS

$1 \leq N \leq 10^5$. $0 \leq \text{start}_i < \text{end}_i \leq 10^9$. $0 \leq P \leq 10^6$.

SAMPLE INPUT

```

words = (0,200) (210,400) (420,600)
(1500,1700) (1705,1900)
P = 100

```

SAMPLE OUTPUT

```

0 600
1500 1900

```

APPROACH

Single left-to-right scan, carrying the current utterance's start and last end. When the gap to the next word exceeds P , close the current utterance and open a new one.

REFERENCE SOLUTION (PYTHON)

```
def merge utterances(words, P):
    utterances = []
    cur_start, cur_end = words[0]
    for start, end in words[1:]:
        if start - cur_end <= P:
            cur_end = end
        else:
            utterances.append((cur_start, cur_end))
            cur_start, cur_end = start, end
    utterances.append((cur_start, cur_end))
    return utterances
```

Problem 4

MEDIUM-HARD

8 MARKS

NORM-1: Currency Normalization for TTS

A TTS front-end must expand Indian Rupee amounts before phonemization. Given a transcript string containing zero or more tokens of the exact form Rs<amount> (a positive integer up to 999999, optionally comma-separated, e.g. Rs1,250), replace each token with its English words followed by 'rupees', using standard thousand/hundred number naming (no 'and'). Leave the rest of the string -- punctuation, spacing -- unchanged.

CONSTRAINTS

1 <= length of transcript <= 2000 characters. 1 <= amount <= 999999.

SAMPLE INPUT

The total is Rs1,250 only.

SAMPLE OUTPUT

The total is one thousand two hundred fifty rupees only.

APPROACH

1) Regex-match Rs[0-9,]+ tokens. 2) Strip commas, parse to int. 3) Convert to words by splitting into a thousands group and a 0-999 group, each built from ones/teens/tens lookup tables. 4) Append 'rupees' and substitute back into the string.

REFERENCE SOLUTION (PYTHON)

```
import re
ONES = ["", "one", "two", "three", "four", "five", "six", "seven", "eight", "nine", "ten",
        "eleven", "twelve", "thirteen", "fourteen", "fifteen", "sixteen", "seventeen",
        "eighteen", "nineteen"]
TENS = ["", "", "twenty", "thirty", "forty", "fifty", "sixty", "seventy", "eighty", "ninety"]

def two_digit(n):
    if n < 20: return ONES[n]
    return (TENS[n//10] + (" " + ONES[n%10] if n%10 else "")).strip()

def three_digit(n):
    if n >= 100:
        rest = two_digit(n % 100)
        return ONES[n//100] + " hundred" + (" " + rest if rest else "")
    return two_digit(n)

def number_to_words(n):
    if n == 0: return "zero"
    thousands, rest = divmod(n, 1000)
```

```

parts = []
if thousands: parts.append(three_digit(thousands) + " thousand")
if rest or not parts: parts.append(three_digit(rest))
return " ".join(p for p in parts if p)

def normalize(text):
def repl(m):
amount = int(m.group(1).replace(",",""))
return number_to_words(amount) + " rupees"
return re.sub(r"Rs([0-9,]+)", repl, text)

```

Problem 5

EASY

8 MARKS

FSM-1: Turn-Taking State Machine

A voice agent's turn-taking logic is a finite state machine with states IDLE, LISTENING, THINKING, SPEAKING and transitions: IDLE-SPEECH_START->LISTENING, LISTENING-SPEECH_END->THINKING, THINKING-TTS_START->SPEAKING, SPEAKING-TTS_END->IDLE, SPEAKING-BARGE_IN->LISTENING. Any event not listed for the current state is ignored. Starting in IDLE, given N events in order, print every state the machine enters (only when it actually changes), one per line -- do not print the initial IDLE.

CONSTRAINTS

1 <= N <= 10⁵. Each event is one of SPEECH_START, SPEECH_END, TTS_START, TTS_END, BARGE_IN.

SAMPLE INPUT

```

SPEECH_START SPEECH_END TTS_START BARGE_IN
SPEECH_END TTS_START TTS_END

```

SAMPLE OUTPUT

```

LISTENING
THINKING
SPEAKING
LISTENING
THINKING
SPEAKING
IDLE

```

APPROACH

Encode the table as a dict keyed by (state, event) -> next_state. Walk the events; on a hit, append the next state and update; on a miss, ignore and keep the current state.

REFERENCE SOLUTION (PYTHON)

```

TRANSITIONS = {
("IDLE", "SPEECH_START"): "LISTENING",
("LISTENING", "SPEECH_END"): "THINKING",
("THINKING", "TTS_START"): "SPEAKING",
("SPEAKING", "TTS_END"): "IDLE",
("SPEAKING", "BARGE_IN"): "LISTENING",
}

def run(events):
state, out = "IDLE", []
for e in events:
nxt = TRANSITIONS.get((state, e))
if nxt is not None:
state = nxt

```

```
out.append(state)
return out
```

Built for the rvhack team's Voice AI Round 1 prep. Pair this with the interactive, timed version (Signal Check) linked in the project README for a closer simulation of the real assessment conditions.